

KubeCon EU 2027 — Talk 2: The Incident We Held Open

Contents

Talk 2 — Full package: “Held Open on Purpose: One Incident, and the Pipeline We Rebuilt Because of It”	1
1. Title (Title Case, inclusive-language checked)	1
2. Description (problem → contribution → relevance)	1
3. Answers to the CFP’s “Preparing to Submit” questions	2
4. Detailed 25-minute talk outline	2
5. Notes for reviewers / “is this a case study” field	3
6. Relevant CNCF / ecosystem projects	3
7. Speaker bio (draft — same as talk 1, reuse verbatim)	4
8. One-line pitch (social/preview card)	4
9. Illustrative diagrams / visuals to build	4
10. Before you submit — checklist	4

Talk 2 — Full package: “Held Open on Purpose: One Incident, and the Pipeline We Rebuilt Because of It”

Source: docs/incidents/2026-08-03-forecast-short-term-public-large-sidecar-dropped.md (infra repo), with the follow-through resolution drawn from the repo’s current state (the Sept 2026 CI/CD pipeline overhaul: PR-gated Terraform, plan-time image-existence guard, CI lint/type/self-test coverage). Not yet in the book — this is new material.

Target track: Platform Engineering Day (primary) — the drift-detection pattern also fits Observability Day if you’d rather pair it there instead of the AI-agent talk. **Format:** Solo, 25 min. **CFP deadline:** Oct 18, 2026, 23:59 CET. Notifications Dec 14, 2026. Event Mar 15, 2027, Barcelona. Submit at <https://sessionize.com/cncf-hosted-co-located-events-europe-2027/>

1. Title (Title Case, inclusive-language checked)

Primary: *Held Open on Purpose: One Incident, and the Pipeline We Rebuilt Because of It* **Subtitle (if the form has one):** *A Sidecar Dropped by One Second, and the Six-Week Path From Detective Control to Preventive One*

Alternates: - *One Second Apart: How Two Task Definitions Drifted Into a Silent Telemetry Gap - Fix Forward, Never Backward: A CI/CD Case Study in Preventable Drift - The Only Control That Worked Ran Five Days Late — Here’s What We Built Instead*

No gendered, ableist, or violent language in any option.

2. Description (problem → contribution → relevance)

On 2026-07-30, two revisions of the same container task definition were registered one second apart, from one engineer’s laptop. The first carried an observability sidecar. The second — meant to be identical, just resized for a heavier workload — didn’t. For the next four days, a once-daily execution path produced **zero telemetry**, and every alert it fired shipped dashboard links that opened empty. **No runtime alert ever caught it, and none could have:** on a once-daily path, “no telemetry” is indistinguishable from “didn’t run,” so any absence-based alert would drown in false positives.

The only thing that *did* catch it was a weekly check that compares declared configuration against reality — four days late, the maximum its cadence allowed. Instead of quietly patching the one broken revision, we left the incident open, on purpose, as the forcing function for a real pipeline redesign: config in source control, a CI check that fails the build if an instrumented service loses its sidecar, deploys from CI only, event-driven detection instead of weekly. Six weeks later, the redesign shipped for real.

This talk walks through the incident, the “what happened → what would have prevented it” table we built from it, and what actually got implemented — and draws the direct line to Kubernetes’ own version of this problem: sidecar-injecting mutating webhooks, service-mesh injection, and OTel Operator auto-instrumentation all carry the identical “one revision has it, one doesn’t” risk, just injected by a different mechanism.

(≈245 words — trim to fit the live field limit.)

3. Answers to the CFP’s “Preparing to Submit” questions

Same caveat as talk 1 — #1 especially is yours to personalize.

Q1 — What are you hoping to get from your presentation? Direct feedback from people running Kubernetes-native sidecar injection (mutating webhooks, service meshes, the OTel Operator) on whether they’ve built an equivalent declared-vs-observed drift check, or hit the same “can’t alert on absence of a low-frequency signal” wall. Validation — or pushback — on generalizing an ECS-specific incident into a cross-orchestrator pattern. Possibly, collaborators for turning the event-driven detection idea (still unbuilt as of writing) into something concrete.

Q2 — What do you expect the audience to gain from your presentation? A reusable diagnostic question for their own systems: “if this signal went silent, would any alert I have actually fire — or would it look identical to normal?” A concrete worked example of the “what happened → what would prevent it” table technique for turning one incident into a scoped set of pipeline changes, rather than a one-off patch. And an honest, unflattering data point — the fix took six weeks and started with a control that ran five days late — that’s more useful to a room full of engineers than a success story with the messy middle removed.

Q3 — How will your presentation help better the ecosystem? Sidecar injection is everywhere in cloud native — service meshes, the OTel Operator, policy/webhook-based mutation — and every one of those mechanisms can silently diverge the same way this task definition did: an injector applied to one revision or one resource and not its sibling, with no runtime signal that would ever fire without also firing constantly on legitimate low-frequency workloads. This talk hands the ecosystem a named failure class, a working detector pattern for it, and a real (not hypothetical) example of turning that detector’s four-day lag into an event-driven, sub-minute one.

4. Detailed 25-minute talk outline

[0:00–1:30] Hook. Show the CloudTrail table: two `register-task-definition` calls, one second apart, same engineer, one with a sidecar container block, one without. Land the line: “*One second apart means one script, or one copy-paste. Nobody sat down and decided to skip the sidecar.*”

[1:30–3:00] What these two revisions power, and why a sidecar at all. One paragraph on why the observability data (metrics/traces/logs) for this container rides in a sidecar rather than being baked into the app image — and one sentence bridging to the room: this is the identical shape as a service-mesh sidecar or an OTel Operator auto-instrumentation injection, just added by hand here instead of by a mutating webhook.

[3:00–5:00] The mechanism. Two independently hand-maintained JSON files, no source control, no review, no pipeline. Onboarding added the sidecar block to one file. The other file simply never received the edit — nothing compared them.

[5:00–7:00] Why four days, and why it couldn’t have been faster at runtime. The core insight of the whole talk: a once-daily path producing zero telemetry looks *exactly* like a once-daily path that didn’t run. Any alert built on “absence of signal” would false-positive constantly on legitimate low-frequency jobs. State plainly: **runtime monitoring was structurally incapable of catching this.**

[7:00–9:00] The detector that did catch it. A weekly job comparing *declared* configuration (what’s registered) against expectations — not a Grafana query, no telemetry credentials needed, immune to query-window artifacts. Two rule codes as two lenses on one fact: did a newer revision regress relative to an older one, and does the currently-advertised revision match what’s actually running. Detection latency: 4 days, bounded by the weekly cadence — worst case 7.

[9:00–10:30] The decision to leave it open. Read the actual incident-doc status line aloud: *“Open by decision — held as the worked example for the upcoming CI/CD process work. Do not close it silently.”* Name why this is unusual and worth copying: most teams patch the one broken revision and move on, spending the incident’s only real value — the fact that it’s fresh, concrete, and already has a willing sponsor — on nothing.

[10:30–15:00] The table, walked through live. Present the “what happened → what would have prevented it” mapping row by row: one source of truth instead of two drifting files; a CI check that fails the build if an `_OTEL_INSTRUMENTED` family’s newest revision has no sidecar; deploy from CI only, human role reduced to approving a diff; PR review where “`-otel-sidecar`” is one visible line; event-driven detection instead of a weekly sweep; documented hazards turned into asserted tests, not paragraphs; deferred decisions given an expiry instead of going stale silently.

[15:00–19:00] Six weeks later: what actually shipped. This is the payoff the incident doc itself doesn’t have yet — bring the audience current. Walk through, mapped explicitly back to table rows: infra Terraform moved to feature-branch + PR, a bot posting the plan on every PR; a plan-time guard (a data source that reads the container image registry) that fails the *plan* — before any apply — if the image the change references doesn’t exist yet, closing exactly the “config references a thing that isn’t there and nobody notices until the rollback” failure shape; CI now running lint, type checks and self-tests over the infrastructure’s own supporting code, not just `terraform plan`. Be candid about what’s still open: event-driven, sub-minute detection (row 5) hadn’t shipped as of the last check — this is a live, ongoing effort, not a tidy resolved case study, and that’s worth saying directly.

[19:00–21:30] The Kubernetes parallel. Make explicit what’s been implicit throughout: a mutating admission webhook that injects a sidecar, a service mesh’s automatic proxy injection, the OTEL Operator’s instrumentation CRDs — every one of these can suffer the identical “this resource has it, that one doesn’t” drift, for the same reason (the injection point and the thing it’s injected into are maintained/applied independently, and nothing reconciles them). The detection pattern (compare declared/expected state against actual running state, on a schedule tight enough to matter) transfers directly, regardless of which mechanism does the injecting.

[21:30–23:30] The honest caveat. *“The only control that worked was one we wrote ourselves, and it ran five days late. Everything preventive was absent.”* Don’t sand this down — it’s the most credible line in the talk, and it’s what makes the six-weeks-later payoff land as earned rather than tidy.

[23:30–25:00] Takeaways and close. Three things attendees can do Monday: (1) ask “would an absence of this signal actually alert, or look normal?” about your own low-frequency jobs; (2) when you find a good incident, consider *not* closing it quietly — use it as the concrete, sponsored case for the pipeline change it justifies; (3) turn every documented hazard into an assertion in code, because a paragraph that says “check for this” is not a control. Close on the title line.

5. Notes for reviewers / “is this a case study” field

Yes — production case study, told with an unresolved middle and a real (verifiable, dated) resolution that shipped independently of this talk being written. Company name, internal domains and resource identifiers generalized. Not previously presented at any Linux Foundation event.

6. Relevant CNCF / ecosystem projects

- OpenTelemetry (the sidecar being injected, and the OTEL Operator as the direct Kubernetes analogue of the injection mechanism this talk is about)
- The general sidecar-injection pattern used by service meshes (mention by category, not brand, unless you want to name one specifically once you pick your framing)
- Kubernetes (as the platform the drawn parallel targets — even though the incident itself runs on ECS Fargate, generalized per the book’s existing anonymization convention)

7. Speaker bio (draft — same as talk 1, reuse verbatim)

Predrag Mujkovic is a Senior DevOps/SRE engineer with close to a decade of experience across cloud and on-premises infrastructure, currently running observability for a weather-data analytics platform on AWS (ECS Fargate, Aurora/RDS, Terraform) with OpenTelemetry and the Grafana LGTM stack (Loki, Grafana, Tempo, Mimir). He is the author of a full-length book documenting that platform's observability implementation from the ground up, based on the real production system.

8. One-line pitch (social/preview card)

Two container task definitions, registered one second apart from a laptop — one with an observability sidecar, one without. It took four days to notice and six weeks to fix properly. Here's the incident we deliberately didn't close, and the pipeline we rebuilt because of it.

9. Illustrative diagrams / visuals to build

Nothing exists yet in the book for this one (it's new material) — worth building before the talk: - The CloudTrail timeline table itself (two rows, timestamps a second apart, sidecar column highlighted) — this is your strongest single slide, use it as the cold open. - A simple before/after of the pipeline: hand-maintained JSON on a laptop → registered directly to production, versus PR → CI plan-time image check → merge → apply. - The “what happened → what would prevent it” table itself, revealed row by row as you narrate it, is strong enough to be a slide directly.

10. Before you submit — checklist

1. Get explicit sign-off (yours, since this is your employer's real incident) on exactly how much operational detail is comfortable to generalize beyond what the book already does — this incident hasn't been through the book's anonymization pass yet.
2. Confirm actual title/abstract character limits on the live Sessionize form.
3. Decide whether to submit this to Platform Engineering Day only, or also flag it for Observability Day (the drift-detection half fits both) — check whether the form allows multiple target events.
4. Build the two/three visuals in §9 before the talk — none exist yet.